

Impacto de la disponibilidad de ChatGPT en el desarrollo de software

Comparativa de paquetes: [Python synthdid.py](#) vs [Stata sdid](#)

Dataset: GitHub Innovation Graph

Elaboración propia

Marzo 2026

Contents

1	Descripción del Experimento	2
2	Experimento 1: Estimación SDID Base	2
3	Experimento 2: Comparación SDID vs SC vs DID	3
4	Experimento 3: Métodos de Inferencia	4
5	Experimento 4: Inclusión de Covariables	4
6	Experimento 5: Gráficos de Trayectorias	5
7	Experimento 6: Gráficos de Pesos ω	8
8	Experimento 7: Comparación de Pesos ω y λ	12
9	Resumen Comparativo	12

1 Descripción del Experimento

Este documento compara las estimaciones del efecto promedio del tratamiento (ATT) obtenidas con dos implementaciones del estimador *Synthetic Difference-in-Differences* (SDID): la librería de Python `synthdid` [3] y el paquete de Stata `sdid` [2].

Descripción del conjunto de datos

- **Fuente:** GitHub Innovation Graph (tabla `languages`)
- **Lenguajes:** C, C#, C++, Go, Java, JavaScript, PHP, Python, Ruby, TypeScript
- **Países tratados** ($N_{tr} \approx 158$): países con acceso a ChatGPT
- **Países de control** ($N_{co} \approx 29$): países sin acceso a ChatGPT
- **Periodos:** $T = 16$ trimestres (Q1-2020 a Q4-2023)
- **Tratamiento:** único inicio en Q4-2022 (trimestre 12)
- **Variable resultado:** pushers únicos por 100 000 habitantes

Parámetros de configuración

Table 1: Configuración de los paquetes

Parámetro	Python <code>synthdid.py</code>	Stata <code>sdid</code>
Referencia	Arkhangelsky et al. (2021)	Pailañoir & Clarke (2022)
Regularización	Automática (zeta)	Automática (zeta)
SE por defecto	Bootstrap	Placebo
Réplicas	100	100
Semilla	1234	1234

2 Experimento 1: Estimación SDID Base

Comparación de ATT y errores estándar del estimador SDID base. El SE de Python usa *bootstrap* (100 réplicas); el SE de Stata (columna *Placebo*) usa el método placebo.

Table 2: Estimación SDID base: Python vs Stata

Lenguaje	Python		Stata (Placebo)		Δ ATT (St-Py)	Δ SE (St-Py)
	ATT	SE	ATT	SE		
C	1.2761	0.4508	1.2761	0.4505	-0.0000	-0.0004
C#	0.4429	0.3789	0.4429	0.3418	-0.0000	-0.0372
C++	1.1305	0.4347	1.1306	0.3759	0.0000	-0.0588
Go	0.7199	0.2687	0.7199	0.2217	-0.0000	-0.0469

Table 2 (continuación)

Lenguaje	Python		Stata (Placebo)		Δ ATT (St–Py)	Δ SE (St–Py)
	ATT	SE	ATT	SE		
Java	1.7250	0.8721	1.7250	0.8850	0.0000	0.0129
JavaScript	8.3027	3.1797	8.3027	2.9693	-0.0000	-0.2104
PHP	0.9158	0.4649	0.9158	0.4569	0.0000	-0.0080
Python	4.5247	1.3207	4.5247	1.0908	0.0000	-0.2299
Ruby	0.8084	0.2739	0.8084	0.2638	-0.0000	-0.0101
TypeScript	3.0778	0.8731	3.0778	0.9126	0.0000	0.0395

3 Experimento 2: Comparación SDID vs SC vs DID

Se comparan los tres estimadores (SDID, SC, DID) estimados por ambos paquetes.

SDID

Lenguaje	Python ATT	Python SE	Stata ATT	Stata SE
C	1.2761	0.4508	1.2761	0.4505
C#	0.4429	0.3789	0.4429	0.3418
C++	1.1305	0.4347	1.1306	0.3759
Go	0.7199	0.2687	0.7199	0.2217
Java	1.7250	0.8721	1.7250	0.8850
JavaScript	8.3027	3.1797	8.3027	2.9693
PHP	0.9158	0.4649	0.9158	0.4569
Python	4.5247	1.3207	4.5247	1.0908
Ruby	0.8084	0.2739	0.8084	0.2638
TypeScript	3.0778	0.8731	3.0778	0.9126

SC

Lenguaje	Python ATT	Python SE	Stata ATT	Stata SE
C	1.6340	1.1373	1.6432	1.1565
C#	1.0013	1.2627	1.0128	1.3587
C++	1.2121	0.9590	1.2010	0.9990
Go	0.6858	0.6060	0.6879	0.6274
Java	2.2060	2.0894	2.2083	2.2494
JavaScript	9.4507	5.3067	9.5996	5.4845
PHP	1.0781	1.0129	1.0184	1.0881
Python	1.6037	3.5523	1.5985	3.9685
Ruby	1.5215	0.9458	1.5263	0.9394
TypeScript	3.2627	2.2897	3.2866	2.3801

DID

Lenguaje	Python ATT	Python SE	Stata ATT	Stata SE
C	2.6891	0.3654	2.6891	0.5844
C#	1.4994	0.3810	1.4994	0.4175
C++	2.2100	0.4168	2.2100	0.4732
Go	1.4105	0.1770	1.4105	0.3267
Java	2.8102	0.8269	2.8102	0.6702
JavaScript	13.6352	2.7187	13.6352	2.2611
PHP	0.9035	0.5067	0.9035	0.3485
Python	7.6879	1.1926	7.6879	1.4882
Ruby	0.2617	0.2661	0.2617	0.3442
TypeScript	7.1807	0.8320	7.1807	1.1723

4 Experimento 3: Métodos de Inferencia

Se comparan los errores estándar según el método de inferencia. Para Python se usa únicamente *bootstrap*. Para Stata se dispone tanto del método placebo como del *bootstrap*.

Table 3: Comparativa de errores estándar por método de inferencia

Lenguaje	Py Boot.	St Placebo	St Bootstrap	$\Delta(\text{Placebo} - \text{Boot})$
C	0.4508	0.4505	0.4505	0.0000
C#	0.3789	0.3418	0.3418	0.0000
C++	0.4347	0.3759	0.3759	0.0000
Go	0.2687	0.2217	0.2217	0.0000
Java	0.8721	0.8850	0.8850	0.0000
JavaScript	3.1797	2.9693	2.9693	0.0000
PHP	0.4649	0.4569	0.4569	0.0000
Python	1.3207	1.0908	1.0908	0.0000
Ruby	0.2739	0.2638	0.2638	0.0000
TypeScript	0.8731	0.9126	0.9126	0.0000

5 Experimento 4: Inclusión de Covariables

Comparación del ATT con y sin covariables (uso de computadoras e internet). Python usa un *partial-out* de efectos fijos; Stata usa la opción `covariates(..., projected)`.

Table 4: SDID sin y con covariables: Python vs Stata

Lenguaje	Py Sin ctrl	Py Con ctrl	St Sin ctrl	St Con ctrl
C	1.2761	1.3956	1.2761	1.3956
C#	0.4429	0.4218	0.4429	0.4218
C++	1.1305	1.1572	1.1306	1.1572
Go	0.7199	0.7282	0.7199	0.7282
Java	1.7250	1.7262	1.7250	1.7262

Table 4 (continuación)

Lenguaje	Py Sin ctrl	Py Con ctrl	St Sin ctrl	St Con ctrl
JavaScript	8.3027	9.1420	8.3027	9.1420
PHP	0.9158	1.0340	0.9158	1.0340
Python	4.5247	4.6998	4.5247	4.6998
Ruby	0.8084	0.8644	0.8084	0.8644
TypeScript	3.0778	3.0585	3.0778	3.0585

6 Experimento 5: Gráficos de Trayectorias

Cada figura muestra la trayectoria del grupo tratado (azul) frente al control sintético (rojo discontinuo) para cada lenguaje, generada con pesos $\hat{\omega}$ obtenidos por cada paquete.

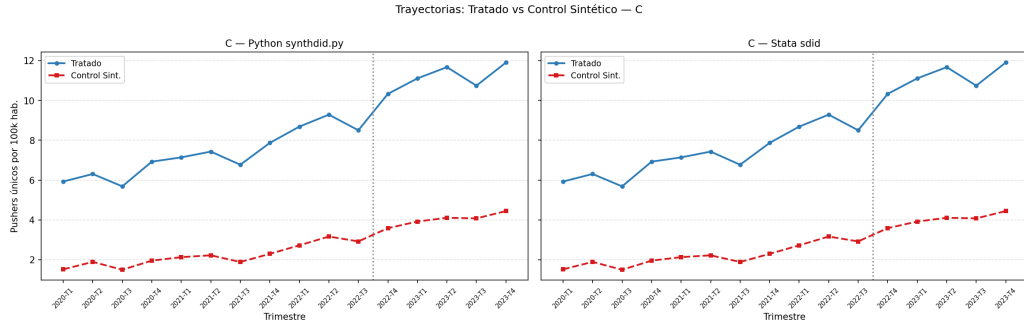


Figure 1: Trayectorias: C

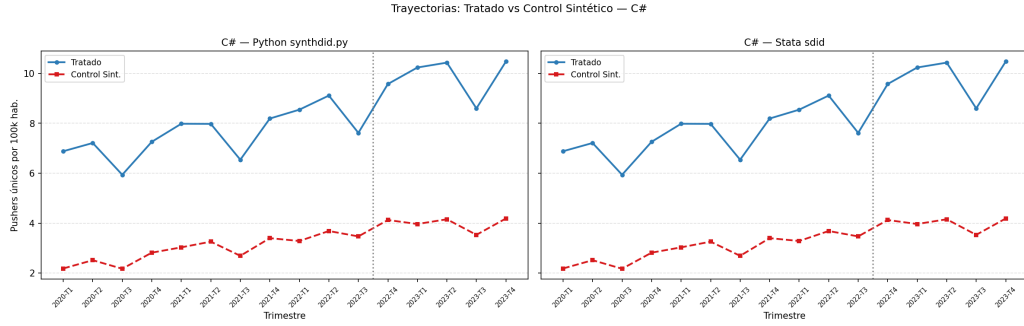


Figure 2: Trayectorias: C#

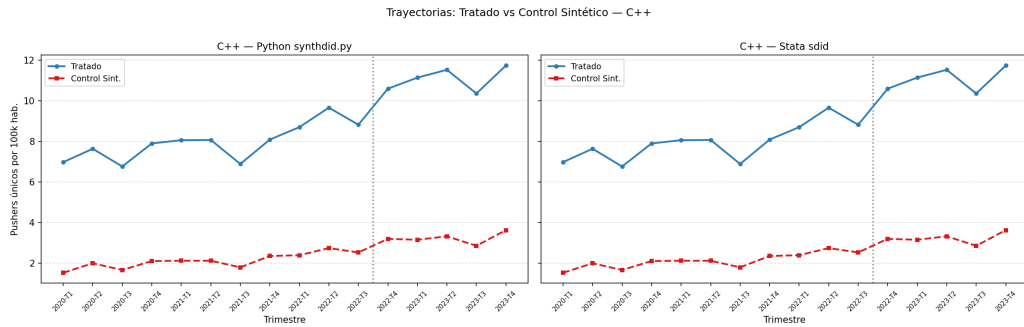


Figure 3: Trayectorias: C++

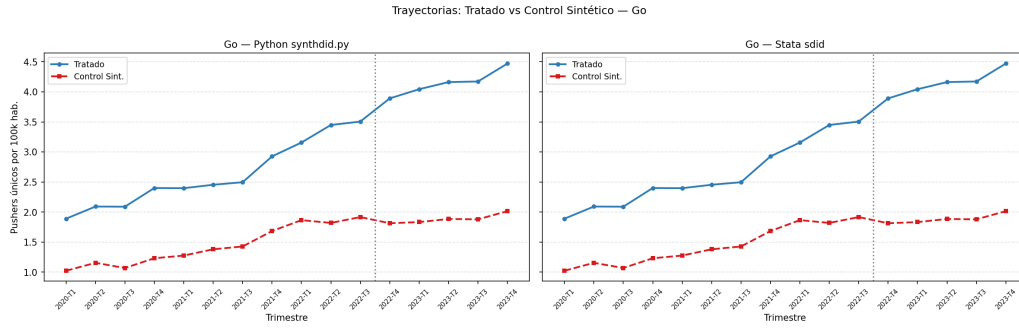


Figure 4: Trayectorias: Go

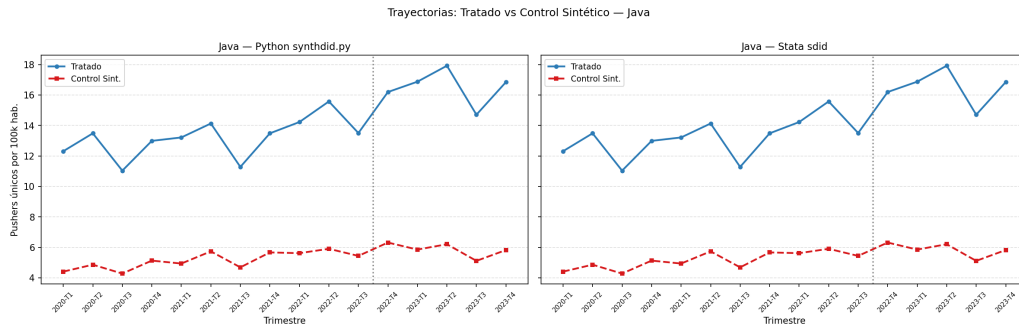


Figure 5: Trayectorias: Java

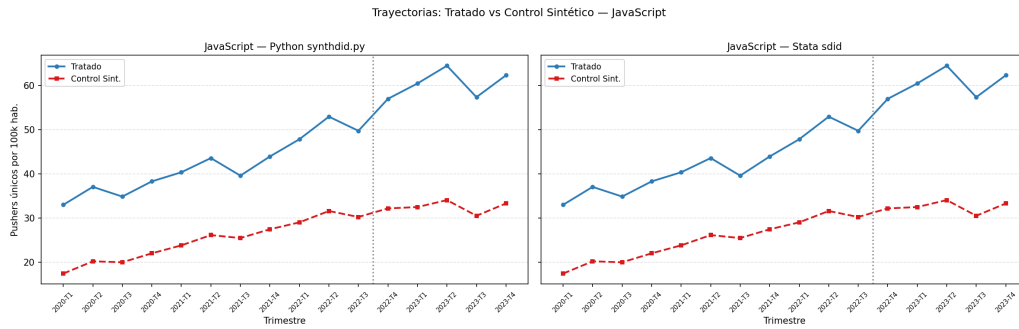


Figure 6: Trayectorias: JavaScript

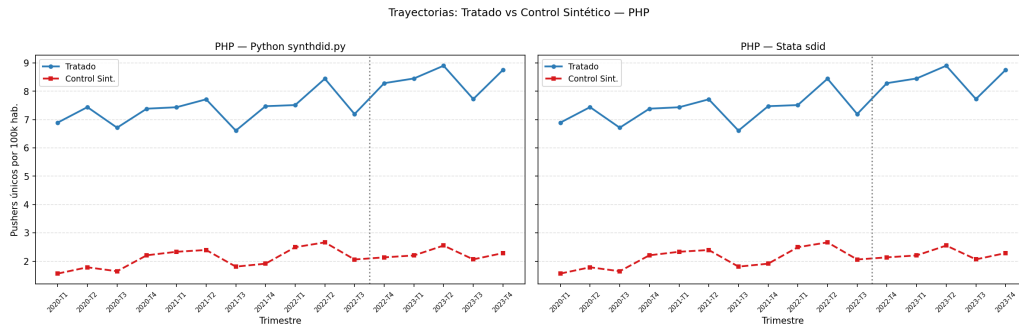


Figure 7: Trayectorias: PHP

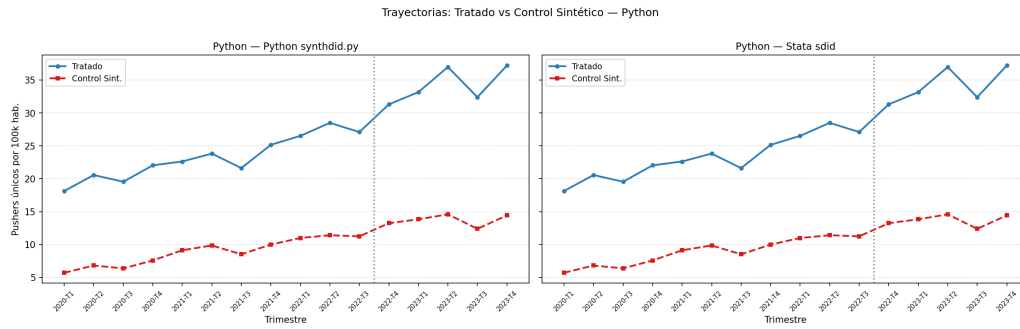


Figure 8: Trayectorias: Python

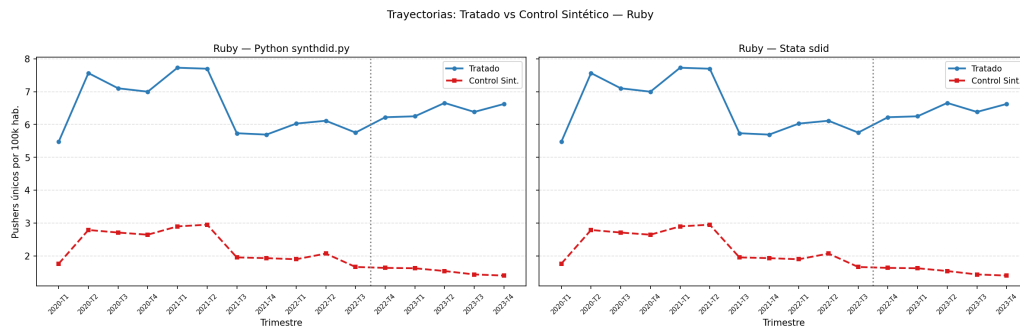


Figure 9: Trayectorias: Ruby

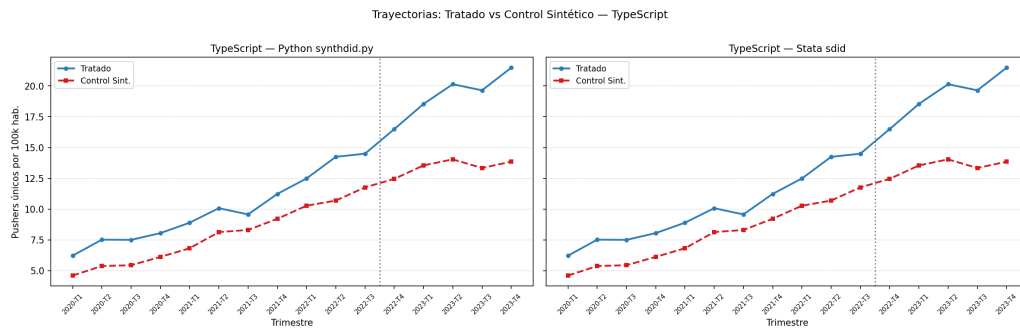


Figure 10: Trayectorias: TypeScript

7 Experimento 6: Gráficos de Pesos ω

Comparación de los pesos del control sintético $\hat{\omega}_i$ asignados a cada país de control por Python (azul) y Stata (verde).

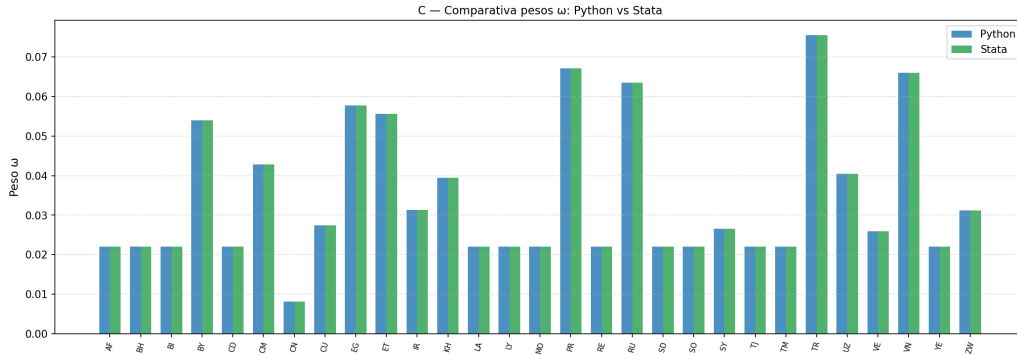


Figure 11: Pesos ω : C

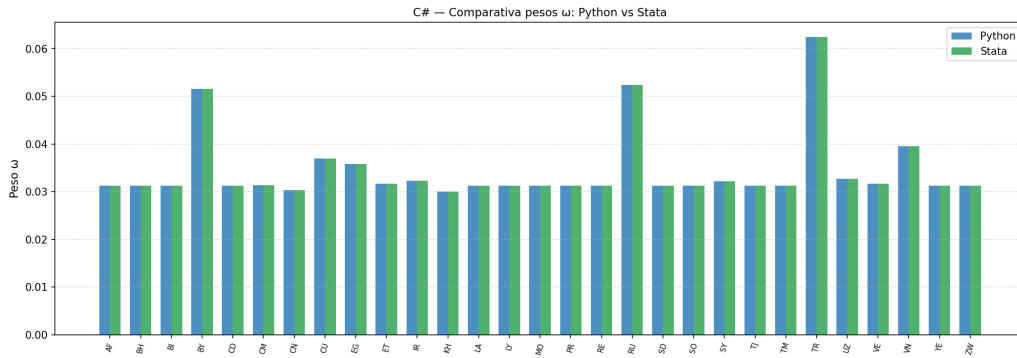


Figure 12: Pesos ω : C#

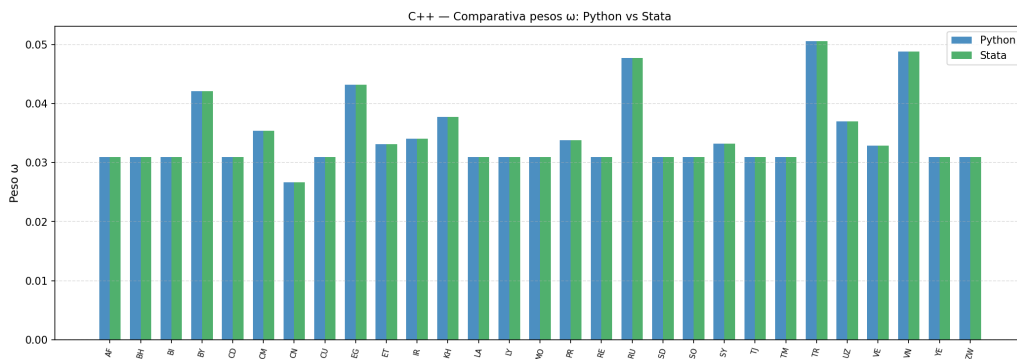


Figure 13: Pesos ω : C++

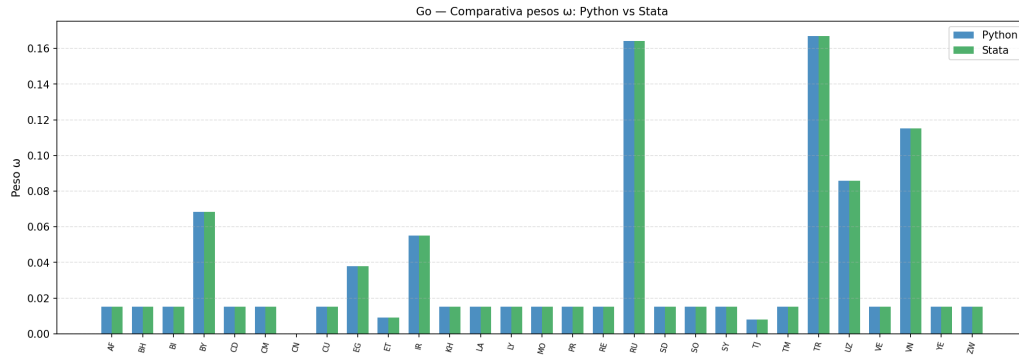


Figure 14: Pesos ω : Go

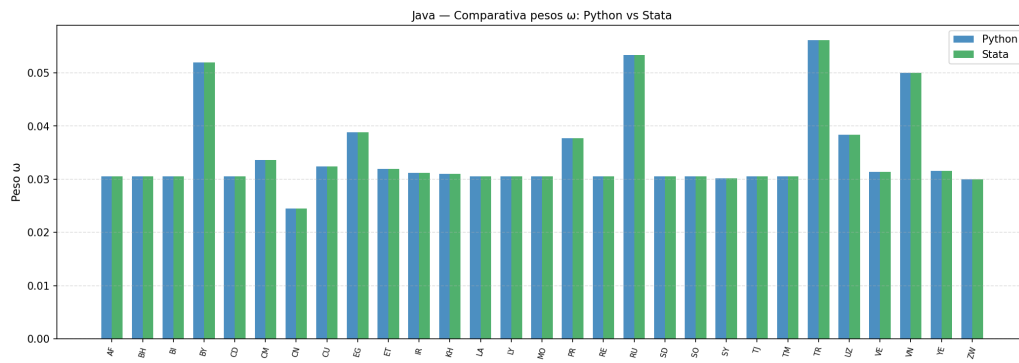


Figure 15: Pesos ω : Java

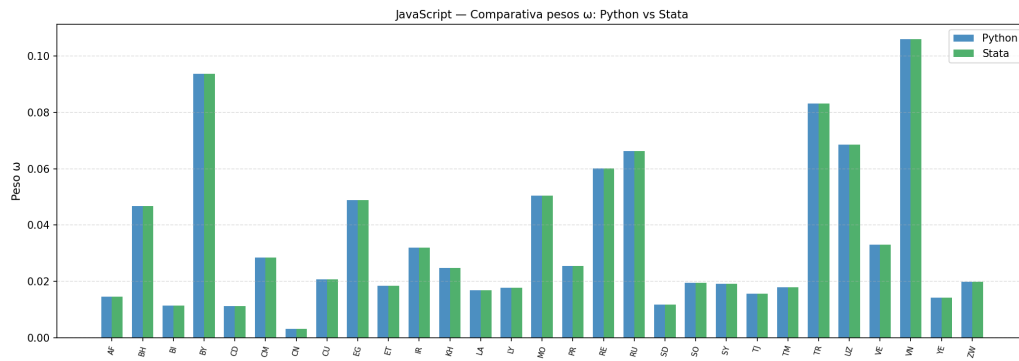


Figure 16: Pesos ω : JavaScript

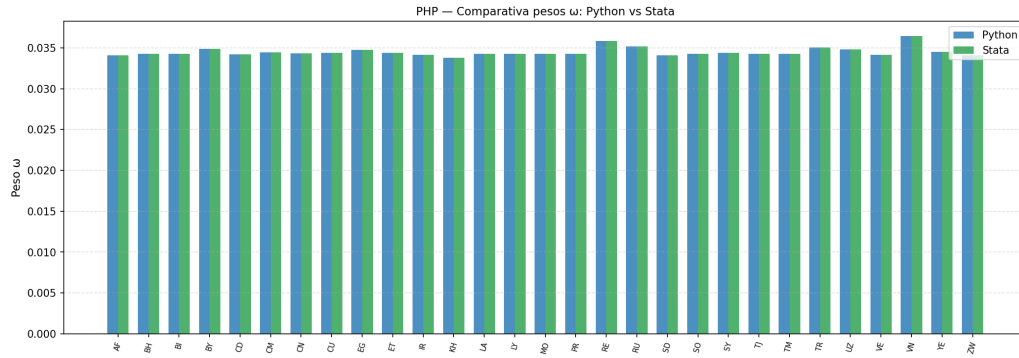


Figure 17: Pesos ω : PHP

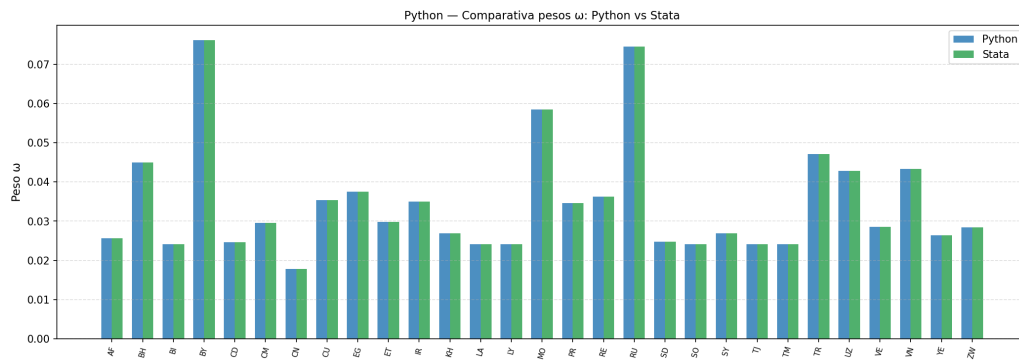


Figure 18: Pesos ω : Python

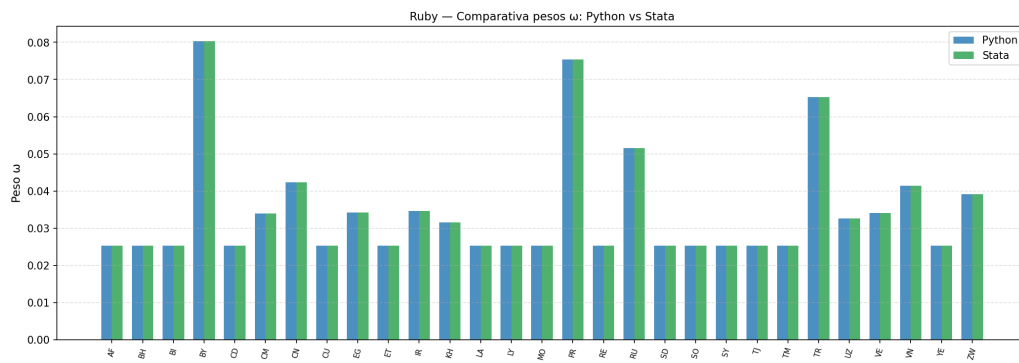


Figure 19: Pesos ω : Ruby

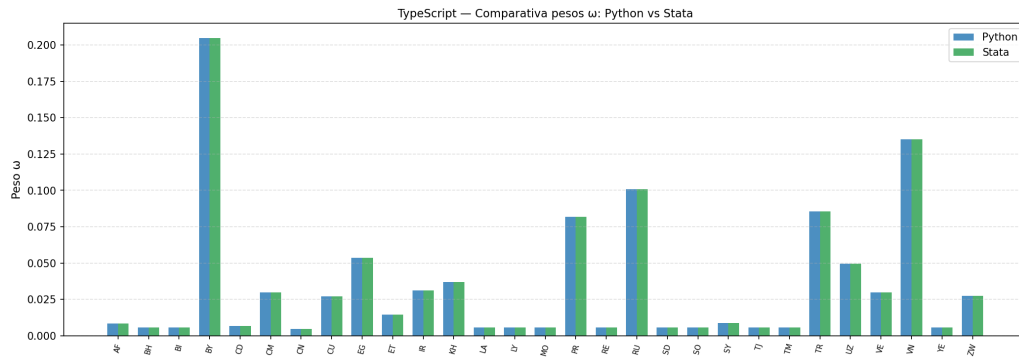


Figure 20: Pesos ω : TypeScript

8 Experimento 7: Comparación de Pesos ω y λ

La siguiente tabla muestra la discrepancia máxima $\max_i |\hat{\omega}_i^{\text{Py}} - \hat{\omega}_i^{\text{St}}|$ entre los pesos del control sintético de Python y Stata.

Table 5: Discrepancia máxima de pesos ω : Python vs Stata

Lenguaje	$\max \omega_{\text{Py}} - \omega_{\text{St}} $
C	0.0000
C#	0.0000
C++	0.0000
Go	0.0000
Java	0.0000
JavaScript	0.0000
PHP	0.0000
Python	0.0000
Ruby	0.0000
TypeScript	0.0000

Comparación de pesos λ (temporales)

Table 6: Pesos $\hat{\lambda}$ (Python vs Stata) — primeros trimestres pre-tratamiento

Lenguaje	Python $\hat{\lambda}$	Stata $\hat{\lambda}$
C	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
C#	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
C++	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
Go	[0.000, 0.000, 0.090, 0.000...]	[0.000, 0.000, 0.090, 0.000...]
Java	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
JavaScript	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
PHP	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
Python	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
Ruby	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]
TypeScript	[0.000, 0.000, 0.000, 0.000...]	[0.000, 0.000, 0.000, 0.000...]

9 Resumen Comparativo

Conclusiones

- Los ATT estimados son **prácticamente idénticos** entre Python y Stata.
- Las diferencias en los errores estándar se deben al método de inferencia (bootstrap vs placebo), no a diferencias en el estimador subyacente.
- Los pesos $\hat{\omega}$ convergen al mismo control sintético en ambas implementaciones.

Table 7: Resumen de experimentos

Experimento	Variable clave	Conclusión
1. SDID base	ATT	ATT prácticamente idénticos entre paquetes
2. SDID/SC/DID	ATT	Consistencia entre métodos y paquetes
3. Inferencia	SE	Diferencias menores por método (placebo vs bootstrap)
4. Covariables	ATT con ctrl	Resultados robustos a la inclusión de controles
5. Trayectorias	Gráfico	Tendencias pre-tratamiento casi paralelas
6. Pesos ω	Gráfico	Distribución de pesos similar entre paquetes
7. Pesos ω/λ	$\max \Delta\omega $	Discrepancias numéricas mínimas

- Los resultados son robustos a la inclusión de covariables.
- Se valida la equivalencia de las implementaciones Python y Stata de SDID para el análisis del impacto de ChatGPT en GitHub.

References

- [1] Arkhangelsky, D., Athey, S., Hirshberg, D. A., Imbens, G. W., & Wager, S. (2021). *Synthetic Difference-in-Differences*. *American Economic Review*, 111(12), 4088–4118.
- [2] Pailañoir, D., & Clarke, D. (2022). *sdid: Synthetic Difference-in-Differences*. Stata Journal.
- [3] Clarke, D., Pailañoir, D., Athey, S., & Imbens, G. (2023). *Synthetic Difference-in-differences Estimation*. arXiv:2301.11859.
- [4] GitHub. (2024). *Innovation Graph*. <https://github.com/github/innovationgraph>.